Viewport Display Modes and Presentation Policy

FemboyEngine2D 0.2.0-alpha.1 ships six viewport display modes, one collision-policy ladder, one render-mask component and one unified light source. Together they answer the questions a 2D scene raises during authoring: is the shipped image correct, is the art laid out correctly, does the collision setup match what the player feels, what costs the frame, which lights reach a sprite, and does the baked distance field agree with the drawing.

This manual is the authored reference for those systems. It describes the exact colour legend of every mode, when to use each one, how to read a bad result and what to change, plus the authoring data behind the diagnostics: the collision mode ladder, render masks, the three light types and their cone behaviour, the static and sleeping-static light lifecycle, time stop, and the SDF primitive palette.

The viewport mode is an editor-environment setting, not a scene setting. It is stored in the project file and mirrored to the editor INI, it never changes a shipped build, and the scene itself is untouched while you switch modes. The two settings keys are `viewport_view_mode` (default `0`, Standard) and `viewport_complexity_budget` (default `8`); the complete key list, its defaults and its clamps are in the [Editor Settings Reference] (EditorSettingsReference_EN.md).

1. Where the controls live

The mode selector is a dropdown in the viewport toolbar, to the left of the Move, Scale, World/Local and arrow-step buttons. The closed control shows the full label of the active mode.

Control	Location	Behaviour
View mode dropdown	Viewport toolbar, left of the gizmo buttons	Lists the six modes in enum order; picking one applies it immediately and prints `view mode: <short label>` in the status bar
Hover help card	Appears only while the closed dropdown is hovered	Title, one-line summary, two to three help paragraphs and the mode colour legend
Viewport mode	Editor Settings panel, Display section	Same six modes as a labelled combo box with the one-line summary as its tooltip
Complexity budget	Editor Settings panel, Display section	Slider from 1 to 64; the normalisation reference of the Shader Complexity heat ramp

The help card is deliberately hover only. It appears next to the cursor after a short delay, it is 480 pixels wide, and it is suppressed while the dropdown list itself is open, because the hover state then belongs to the list rows. Read it when you are unsure which mode to pick; it repeats the same text as this manual.

Three rules make the selector predictable:

- The dropdown order, the keyboard order and the saved value share one enum, so `0` is always Standard and `5` is always the SDF Visualizer.
- New modes are appended, never inserted, so a saved project keeps its meaning across engine versions.
- An out-of-range stored value is clamped to Standard when the project is decoded, so a hand-edited file can never leave the viewport in a blank state.

2. The six modes

Two modes show the scene itself: Standard and Unlit. Four modes are diagnostic overlays: Collision Overlap, Shader Complexity, Light Complexity and the SDF Visualizer. An overlay replaces the sprite colour with a diagnostic tint, so the image tells you about data rather than about art. Switching back to Standard proves whether a fix changed the shipped image or only the overlay.

At a glance:

Dropdown label	Short label	Question it answers	Overlay	Reads
Standard (Lit)	Standard	Is the shipped image correct?	No	Lights
Unlit / Wireframe	Unlit	Is the art, the UV rectangle and the pivot correct?	No	Collider bounds
Collision Overlap	Collision	Does every collider match its intended participation level?	Yes	Colliders
Shader Complexity / Overdraw	Overdraw	Which pixels pay for overlapping fills and lights?	Yes	Lights, colliders
Light Complexity	Lights	How many lights reach each sprite?	Yes	Lights
SDF Visualizer	SDF	Does the baked distance field agree with the drawing?	Yes	SDF field, colliders

2.1 Standard (Lit)

The reference image. Sprites keep their authored colour and every light is applied exactly as a shipped build applies it: ambient fill, global lights, omni lights, spot and arc cones, distance falloff, SDF shadows from the baked field and from dynamic occluders, ambient occlusion and tonemapping.

Legend and exact tints:

Legend row	RGB	Hex	Meaning in the image
Authored sprite colour	180, 190, 210	`#B4BED2`	The sprite albedo before any light is added

Ambient fill	40, 44, 60	#282C3C	The floor every pixel receives even without a lamp
Light contribution	255, 214, 120	#FFD678	The additive result of the lights that reach the sprite
SDF shadow	90, 70, 120	#5A4678	The part of the contribution removed by an occluder or a crease

When to use it:

- As the baseline before and after every change to lights, masks or collision policy. A fix that only looks right in an overlay is not a fix.
- When a sprite is unexpectedly dark, to decide whether the cause is light, shadow or the sprite itself.
- Before capturing screenshots or trailers, because it is the only mode that is also the shipped image.

Reading a bad result:

- A dark sprite usually sits behind an occluder, outside every light radius, or under a low ambient term. Raise the ambient fill, move the lamp, or check the occluder list before you edit the sprite colour.
- A sprite that looks correct here but vanishes in Unlit has a layout problem, not a lighting problem: see the next section.
- A washed-out sprite means its light contribution saturates. Lower intensity, narrow the cone, or replace several decorative glows with one global light.

2.2 Unlit / Wireframe

The authored albedo with the lighting solve removed, plus the bounds of every sprite, skinned mesh and cloth mesh drawn as an outline. Use it to judge texture, UV rectangle, pivot and tint with no ambient fill and no shadow hiding them.

Legend and exact tints:

Legend row	RGB	Hex	Meaning in the image
Unlit sprite	120, 220, 255	#78DCFF	The sprite drawn with its authored colour and no light applied
Sprite bounds	80, 160, 255	#50A0FF	The quad of a regular sprite
Mesh or cloth bounds	60, 220, 180	#3CDCB4	The bounds of a skinned or cloth mesh

When to use it:

- Before tuning lights: verifying UV rectangles, pivots, sprite sizes and tint values is cheaper here than fighting a shadow that hides a layout mistake.
- After importing a new atlas, to see that every frame samples the intended region.

- When a sprite is missing from Standard, to distinguish a layout problem from a lighting problem.

Reading a bad result:

- Missing here and missing in Standard: the RenderMask excludes the sprite pass, the colour alpha is zero, or the entity has no DrawOrder component.
- A blank rectangle where art should be: the sprite alpha is zero, or the UV rectangle samples a transparent region of the atlas.
- Bounds offset from the art: the pivot or the sprite size is wrong, not the texture.
- A mesh outline without art: the skin palette or the vertex count is wrong.

2.3 Collision Overlap

The scene is tinted grey and every collider is drawn with the colour of its collision mode. The overlay reads exactly the `CollisionMode` that the physics bridge applies, so a mismatch between the drawn colour and the behaviour is a real configuration bug, not a visual artefact.

Legend and exact tints:

Legend row	RGB	Hex	Meaning in the image
Simple collider (Box2D)	80, 220, 120	`#50DC78`	A Box2D polygon, static or dynamic
Complex SDF collider	250, 210, 70	`#FAD246`	A Box2D polygon that also tests the SDF field
Trigger or query only	230, 90, 220	`#E65ADC`	A sensor: reports overlaps, never pushes
No collision	140, 140, 150	`#8C8C96`	No policy and no collider, or the NoCollision rung

The colour comes from the entity's `CollisionPolicyComponent`. An entity with a `Collider2D` but no policy is drawn as a simple dynamic collider; an entity with neither is drawn as NoCollision.

When to use it:

- Auditing a level: every green shape must be static geometry or a moving prop, every yellow shape must justify the field test, every magenta shape must be a trigger, and every grey shape must be intentional.
- Debugging "the player walks through a wall" and "the player is pushed by a trigger": the overlay shows which rung the author actually assigned.
- Reviewing a prefab or a spawned wave, where inherited policies are easy to forget.

Optimisation recommendations:

- Keep level geometry on simple static shapes. A complex static collider pays for a distance-field sample on every contact test and rarely changes behaviour.
- Reserve complex dynamic shapes for characters and props that need slope response or thin-wall response.

- Remove NoCollision from sprites that still receive overlap queries: they pay the query cost and can never respond.
- Collapse decorative clusters into one simple static shape instead of dozens of tiny colliders.

The full ladder, including what each rung does to the body and to the shape filter, is in section 4.

2.4 Shader Complexity / Overdraw

An estimate of the per-pixel work a sprite receives, drawn as one continuous green to yellow to red heat ramp. The estimate multiplies the number of sprite quads covering the sprite by the number of lights that reach it, normalised by the complexity budget.

Legend rows and their exact tints:

Legend row	RGB	Hex	Meaning in the image
Low cost	80, 220, 120	`#50DC78`	A single fill with little or no light
Moderate cost	230, 230, 90	`#E6E65A`	Two to three overlapping fills
High cost	250, 150, 60	`#FA963C`	Four or more fills, or fills under several lights
Overdraw hotspot	230, 60, 60	`#E63C3C`	A saturated pixel: the budget is reached or exceeded

The tint is computed, not picked from four buckets, so intermediate colours are normal. The ramp runs green to yellow in its first half and yellow to red in its second half, with the blue channel pinned near 38 so the heat replaces the sprite colour instead of tinting through it. The value is:

```
cost       = overlapping sprite count * (reaching lights + 1)
normalised = cost / viewport_complexity_budget
```

A normalised value of zero maps to pure green, 0.5 to pure yellow and 1.0 or more to pure red. The budget is `viewport_complexity_budget`, default 8, clamped to 1 through 64: raise it to make a scene look cooler, lower it to expose a scene that is heavier than it should be. The budget sets the diagnostic reference; it does not change rendering.

When to use it:

- During performance work on fill-bound scenes: portraits over painted backgrounds, particle curtains, stacked glows, dozens of overlapping decals.
- Before adding one more translucent layer to a scene that already saturates.
- When the GPU is the bottleneck but the draw-call count looks healthy.

Reading a bad result:

- A red full-screen band usually means a background quad sits under every subsequent layer. Split the background into regions or draw it once.

- Red islands under a cluster of lamps mean lighting saturation rather than overdraw. The overlay adds both causes, so check Light Complexity before you cut art.
- A single white sprite that is red everywhere: it overlaps many quads or it carries many lights; use the two complexity modes together to tell which.

Optimisation recommendations:

- Cut overlapping transparent quads; every layer multiplies fill cost.
- Merge small sprites into an atlas so the same frame region is filled once.
- Cull off-screen and background-only items through `RenderMaskComponent`.
- Replace several decorative glows with one global light or one omni light.
- Lower the budget only after the scene is understood; a budget that always shows red teaches nothing.

2.5 Light Complexity

The number of lights that reach each sprite after the radius and cone tests.

Legend and exact tints:

Legend row	RGB	Hex	Meaning in the image
Ambient only	90, 140, 255	`#5A8CFF`	No light reaches the sprite
One light	80, 220, 120	`#50DC78`	Exactly one light reaches the sprite
Two or three lights	230, 230, 90	`#E6E65A`	Two or three lights reach the sprite
Four or more lights	230, 60, 60	`#E63C3C`	Four or more lights: tonemapping starts to compress highlights

Sprites with no light are drawn blue; the rest use the same heat ramp as the overdraw mode, with the light count normalised as `(lights - 1) / 3`. That is why one light is green, two and three are yellow and four or more is red.

The count comes from the authoring data, not from the final colour: it walks the enabled light sources, skips a light whose owner excludes the light-source render pass, applies the radius test, and applies the full cone test for spot and arc lights. Legacy radial lights and global lights participate too, so a scene that mixes the two component generations still reports a truthful count.

When to use it:

- When a sprite is brighter than its neighbours and you suspect too many lights.
- Before adding a lamp to a corridor that already has several.
- When you want to prove that a static lamp is classified correctly and no longer participates in the per-frame solve.

Reading a bad result:

- A static lamp that still behaves like a dynamic one usually has the sleeping-static flag cleared, or the static contribution cache was invalidated by a moving target or a moving occluder. The counters described in section 7 tell which.
- A sprite that is red while the screen looks dim: the lights reach the sprite but their contributions are blocked by shadows. Check the occluders.
- A sprite that is blue inside a lit room: its owner excludes the lighting render pass, or the light's owner excludes the light-source pass.

Note on the count: the overlay counts every enabled light that reaches the sprite, including lights marked static, and the in-viewport help card states the same. The static contribution cache affects the per-frame solve, not this diagnostic: use the overlay to compare sprites with each other, and treat it as a measure of authoring load rather than a promise about draw-call counts.

Optimisation recommendations:

- Mark lamps Static so their contribution is cached until the scene changes.
- Reduce Radius; a lamp that lights three rooms costs three rooms.
- Narrow the cone of spot and arc lights. A full-turn omni light and a narrow arc cost the same solve, but the arc reaches fewer targets and is easier to balance.
- Prefer sleeping static props and lights that wake only on contact or on a program trigger.

2.6 SDF Visualizer

The signed distance to the nearest solid surface, sampled at every sprite, with the baked occluders drawn on top.

Legend and exact tints:

Legend row	RGB	Hex	Meaning in the image
Solid (inside the field)	230, 60, 60	`#E63C3C`	The sample point is inside solid geometry
Surface band	250, 210, 70	`#FAD246`	Within 16 world units of the surface
Open space	80, 210, 220	`#50D2DC`	Far from any surface, or no valid field
Baked occluder	160, 120, 255	`#A078FF`	A baked occluder outline that drives shadows and SDF collision

The surface band is fixed at 16 world units. A sprite drawn cyan in a corridor means the field has no valid data there or the sample is far from every surface; a sprite drawn red inside a room means the field is shifted or too coarse.

When to use it:

- Before trusting shadows: a field that disagrees with the art produces shadows that look like bugs.
- When enabling complex collision on a slope, a ramp or a thin wall.

- After editing level geometry, to confirm that the field was rebuilt.
- When tuning the field cell size: coarse cells make the yellow band blocky and the red interior thick.

Reading a bad result:

- Red where the art shows open floor: the field is shifted, or the artist edited the terrain after the bake.
- Cyan where the art shows a wall: the field was not rebuilt after the edit, or the wall is not tagged as an occluder.
- A blocky surface band: the cell size is too coarse for the geometry.
- No colour variation at all: the presenter received no valid field.

Optimisation recommendations:

- Increase the cell size on large open maps; the field is a baked asset and the resolution is a memory and accuracy trade-off.
- Rebuild the field only after edits, not every frame.
- Keep static terrain in the baked field and let moving props use dynamic occluder components instead of forcing a rebuild.

3. Choosing a mode, and the order of a review

A productive review walks from the data to the image, not the other way round:

1. Collision Overlap: every collider must have the participation level you intended before anything else matters.
2. SDF Visualizer: the field must agree with the drawing before complex collision or shadows can be trusted.
3. Unlit / Wireframe: the art, UVs and pivots must be right before light can be judged.
4. Light Complexity and Shader Complexity: find the sprites and pixels that cost more than they should.
5. Standard: confirm that the fix changed the shipped image.

A short table for the common questions:

Symptom	First mode	Then
A sprite is too dark or too bright	Standard	Unlit for the albedo, Light Complexity for the count
The player falls through a wall	Collision Overlap	SDF Visualizer if the wall is a complex collider
Shadows look wrong	SDF Visualizer	Standard to judge the result
The frame is GPU-bound	Shader Complexity	Light Complexity to separate lighting from overdraw
A trigger pushes the player	Collision Overlap	Check the magenta shapes against the trigger list
A lamp does not react	Light Complexity	Check the static flags and the cache counters

4. The collision-mode ladder

Collision participation is authored with `CollisionPolicyComponent`, not with the shape. The policy is separate from `Collider2D` so a designer can change the participation level without rewriting the shape or the layer matrix, and so a snapshot keeps both.

Field	Type	Default	Meaning
`Mode`	uint32	4 (SimpleDynamic)	The ladder rung, stored as a number so TOML, Mirror and the inspector need no enum codec
`Category`	uint32	1	The Box2D category bits of the shape
`Mask`	uint32	0xFFFFFFFF	The Box2D mask bits; written as zero for NoCollision
`Flags`	uint32	0	Reserved authoring bits

The seven rungs, in increasing participation:

Rung	Value	Body	Shape filter	Sensor	Field test	Contact events	Typical use
NoCollision	0	Authored type kept	Mask forced to zero	No	No	No	Purely decorative art, probes that must fall through
QueryOnly	1	Authored type kept	Category and Mask from the policy	Yes	No	Yes	Overlap and line-of-sight queries, pickups
TriggerOnly	2	Authored type kept	Category and Mask from the policy	Yes	No	Yes	Gameplay trigger volumes
SimpleStatic	3	Static	Category and Mask from the policy	No	No	Yes	Walls, floors, one-way platforms
SimpleDynamic	4	Dynamic	Category and Mask from the policy	No	No	Yes	Crates, enemies, the player
ComplexStatic	5	Static	Category and Mask from the policy	No	Yes	Yes	Slopes, thin walls, exact terrain
ComplexDynamic	6	Dynamic	Category and Mask from the policy	No	Yes	Yes	Characters that must follow slopes

How the rungs differ:

- NoCollision never touches the broad phase: the shape filter mask is zero, so no contact is produced, and the authored body type is kept, which lets a collision probe fall through geometry on purpose. It emits no events.
- QueryOnly and TriggerOnly are sensors: they report overlaps, produce no impulse, and emit contact events. The bridge treats the two identically; the ladder keeps them apart so a trigger volume can be found, audited and coloured separately from a query probe. Both are drawn magenta.
- SimpleStatic and SimpleDynamic resolve contacts with the Box2D polygon alone. They are the cheapest rungs and the right default for level geometry and ordinary props.
- ComplexStatic and ComplexDynamic additionally test the baked SDF field, which is what makes slopes, ramps and thin walls behave correctly. They cost more per contact and depend on a rebuilt field.

The policy is applied by `ApplyCollisionPolicyToCollider` before bodies are created, so a mode change never leaves a stale shape filter behind, and a policy-managed body is always enabled: NoCollision removes the shape from the broad phase rather than disabling the body.

Choosing a rung:

6. Start everything as SimpleStatic or SimpleDynamic. Add complex rungs only for surfaces where the polygon is visibly wrong.
7. Use QueryOnly for probes that must not disturb motion, TriggerOnly for volumes that gameplay reacts to.
8. Keep NoCollision for decorative art only, and revisit any grey shape that sits where a player can walk.
9. Re-check the Collision Overlap overlay after adding a new gameplay system: new prefabs inherit whatever policy their template had.

5. Render masks

`RenderMaskComponent` lets one entity opt out of render passes without being disabled or hidden. It is a pair of masks rather than a single one so an authoring preset can exclude a pass and a later override can include it again.

Pass	Bit	Value	Gates
`RenderPassSprites`	0	1	The sprite and skinned-mesh draw of this entity, and therefore its diagnostic tint
`RenderPassLighting`	1	2	Whether this entity receives a light result and participates in the static light cache
`RenderPassShadows`	2	4	Whether this entity's shadow sampling is evaluated
`RenderPassLightSources`	3	8	Whether a light source owned by this entity participates in the solve and the light count

`RenderPassColliders`	4	16	The collider pass of the overlay extract
`RenderPassOverlays`	5	32	Diagnostic overlay primitives owned by this entity
`RenderPassSdf`	6	64	Distance-field contributions owned by this entity
`RenderPassAll`	-	0xFFFFFFFF	Every pass

The rule is simple and total: a pass is allowed when its bit is set in `Include` and clear in `Exclude`. `Exclude` wins when both are set, which makes a late override safe. `ExcludePass` adds a bit to `Exclude`; `IncludePass` adds the bit to `Include` and clears it from `Exclude`.

Field	Type	Default	Meaning
`Include`	uint32	`RenderPassAll`	Passes this entity may appear in
`Exclude`	uint32	`RenderPassNone`	Passes this entity never appears in

Practical recipes:

Goal	Mask
Art invisible in the viewport but still simulated	Exclude Sprites
Art visible but never lit, for an unlit UI layer drawn in the world	Exclude Lighting
A light that is authored but does not contribute yet	Exclude LightSources on the light entity
A sprite that must not receive shadows or cast them	Exclude Shadows
A diagnostics-only helper shape	Exclude everything except Overlays

Because the mask is a plain pair of 32-bit integers, it serialises to TOML, snapshots and the inspector without a custom codec, and a prefab can carry a different default from its instances.

6. Lights: one component, three shapes

`LightSourceComponent` covers every 2D light shape. The type decides how the outer aperture is interpreted; the remaining fields are shared.

Type	Value	Position and radius	Cone	Notes
Global	0	Ignored	Ignored	Adds its colour and intensity to the base term, after ambient; no falloff, no per-target shadow
Omnidirectional	1	Used	Forced to a full turn regardless of `ConeAngle`	The ordinary lamp
SpotArc	2	Used	Uses `ConeAngle` and	Narrow aperture is a spotlight, wide

			`ConeSoftness`	and soft aperture is an arc lamp

The solved light fields:

Field	Type	Default	Meaning
`Color`	Color32	White	Light colour; red, green and blue are unpacked per channel
`Radius`	float	256	World units where the contribution reaches zero; a global light ignores it, and the solver treats values below 1 as 1
`Intensity`	float	1	Multiplier after the two attenuation terms; clamped to zero and above by the solver
`Offset`	Vector2D	0, 0	Offset added to the owner's transform position
`ConeAngle`	float	6.283185 (full turn)	Full aperture in radians; values above a full turn clamp to it, values at or below zero give no light
`ConeSoftness`	float	0.35	Width of the angular falloff band in radians, clamped to half the aperture
`Falloff`	float	2	Distance exponent: 2 is inverse-square-like, 1 is linear
`Type`	uint32	1 (Omni)	The light type above
`Flags`	uint32	Enabled, Static, Sleeping	Enabled, Static, Sleeping, CastNoShadow, WakeOnContact

The direction of a spot or arc light is the owner's `Transform2D.Rotation`, so rotating the owner aims the cone.

Cone behaviour, exactly:

10. The half aperture is `ConeAngle / 2`. Softness is clamped to that half; an aperture of zero or less returns zero everywhere.
11. An aperture at or above a full turn returns one everywhere, which is why the same function serves omni and cone lights.
12. With the absolute angle between the light direction and the direction to the receiver, an angle at or beyond the half aperture returns zero, an angle inside the unsoftened core returns one, and the band between them uses a smoothstep, so there is no visible ring at the edge of the cone.

Distance behaviour: the contribution is `(1 - distance / Radius)` raised to `Falloff`, and zero beyond the radius. The two limits are different on purpose: a cone is a hard cut with a soft band, a radius is a continuous falloff.

What the solver does with those values:

Step	Behaviour
Ambient	The ambient colour times ambient intensity is the floor
Global lights	Added to the base after ambient
Omni, spot and arc lights	Distance times cone times intensity, then the SDF shadow factor if shadows are enabled and a field is active
Occlusion	A distance-to-solid ratio around the target scales the static contribution
Tonemapping	`x / (1 + x)` per channel, so highlights compress instead of clipping to white
Emissive	The target component can lift the result towards white after tonemapping

The solver settings live in `LightSourceSettings`:

Setting	Default	Meaning
`Ambient`	40, 44, 60	The fill colour
`AmbientIntensity`	0.25	The fill strength
`MaxLights`	128	Capacity of the per-frame light sample array
`Tonemap`	true	Apply `x / (1 + x)`
`SdfShadows`	true	Evaluate SDF shadows
`SdfMaxSteps`	16	Marching budget per light-target pair, clamped to 1 through 64
`SdfSoftness`	24	Penumbra width of the shadow test
`SdfAmbientOcclusion`	0.35	Strength of the crease darkening
`SdfOcclusionRadius`	32	Radius of the crease test
`CacheStaticLights`	true	Keep the static contribution between frames

A scene uses one solver for each component generation: `LightSourceSystem` for `LightSourceComponent`, the legacy `LightingSystem` for `Light2DComponent`. Both write the same result component, so the render extract composes the result without knowing which solver produced it.

7. Static lights, dynamic lights and the sleeping-static lifecycle

Light classification is three flags on the component:

Flag	Bit	Meaning
`LightSourceEnabled`	1	The light participates at all
`LightSourceStatic`	2	The contribution is cached until the scene, the targets, the occluders or the field change
`LightSourceSleeping`	4	The static light may wake;

		without the static bit it has no effect
`LightSourceCastNoShadow`	8	The light never evaluates a shadow test
`LightSourceWakeOnContact`	16	A contact may wake a sleeping static light

A light with Static and without Sleeping is permanently static. A light with both bits is a sleeping static light: it contributes through the cache but wakes into the dynamic solve on the first allowed contact or program trigger.

Waking one light is `WakeLightSource`:

- It returns false when the light is not sleeping. A dynamic light cannot be woken twice.
- It returns false when the wake comes from a contact and the wake-on-contact permission is missing.
- Otherwise it clears the Static and Sleeping bits, and the next solve treats the light as dynamic.

The static contribution cache is the reason static lights are cheap. Each frame the solver builds a signature from the data that can change a static result:

- every static light's position, direction, radius, intensity, colour, aperture, softness and falloff;
- every dynamic occluder's centre and radius;
- the shadow softness and ambient occlusion settings;
- the identity of the SDF field;
- every lighting target's entity id and position, because a moving receiver must rebuild even when every light stayed put.

If the signature matches the cached one and caching is enabled, the solver reuses the cached contribution and increments `StaticCacheHitCount`. Otherwise it recomputes every target and increments `StaticRebuildCount`. `InvalidateStaticCache` forces the next solve to rebuild; a scene load, a teleport of a whole room, or a debug toggle are the natural call sites.

The two counters are the cheapest way to prove that the cache works: a corridor with static lamps should show a growing hit count and a rebuild count that stops after the scene settles. A rebuild count that grows every frame means the signature is changing every frame: a moving target, a moving occluder, or a re-created field pointer.

Sleeping static entities are the general form of the same idea, and they are broader than lights: a crate stack, a collidable decoration or a lamp can all be static until something touches them.

Flag	**Bit**	**Meaning**
`kSleepingStaticFlagActive`	1	The component is active
`kSleepingStaticFlagSleeping`	2	The entity is currently static and asleep
`kSleepingStaticFlagWakeOnC`	4	A contact may wake it

ontact		
kSleepingStaticFlagWakeOnTrigger	8	A trigger may wake it
kSleepingStaticFlagLight	16	The owner is also a light source, so waking reclassifies the light

The default flags are active, sleeping and wake-on-contact: scenery that is static until something touches it, which is the authoring state most props want. `WakeCount` and `LastWakeReason` record the history for the inspector and the diagnostics panel. The reasons are None, Contact, Trigger, Program and SceneEdit.

The lifecycle:

State	Event	Result
Sleeping, wake-on-contact	A contact of type BeginTouch or Hit	Sleeping flag cleared, reason Contact, count incremented, body switched to dynamic and awake, light switched to dynamic
Sleeping, wake-on-trigger	A program or trigger wake	Sleeping flag cleared, reason Trigger or Program
Awake	Nothing	It stays awake: waking is one-way until a scene edit
Awake	A scene edit	`ResetSleepingStatic` restores the active and sleeping flags and clears the reason
Dynamic light	`WakeLightSource`	Returns false: nothing to wake

Waking is deliberately one-way, because determinism depends on it: a contact on tick N wakes the entity on every replay of tick N. A simulation that put the entity back to sleep on its own would depend on frame timing and would diverge.

The practical recipe for a lamp-lit room:

13. Author every lamp as Static with the sleeping flag set and wake-on-contact enabled. The room costs nothing per frame after the first solve.
14. Wake only the lamps the player can break or switch, through a program trigger rather than a contact, so the wake does not depend on collision geometry.
15. Give the scenery `SleepingStaticComponent` with the default flags, and let contacts wake it.
16. Watch `StaticRebuildCount`: if it grows while nothing moves, something in the signature moves every frame.

8. Time stop and slow motion

Two independent controls exist, and they multiply.

The global one is `PhysicsSystem::SetTimeScale`. It scales the world step: the step passed to Box2D is the fixed delta times the scale. A scale of one is real time, 0.5 is half speed, 0.25 is bullet time. A scale of zero or below stops the world: the step is skipped entirely, so

nothing integrates and no contact events are produced, while the system still applies presentation policies, syncs bodies and rebuilds missing bodies.

The per-entity one is `TimeScaleComponent`:

Field	Type	Default	Meaning
`Scale`	float	1	Clock multiplier of one entity
`Reserved`	float	0	Reserved

Per-entity scale is applied every physics step to every entity that has both the component and a `RigidBody2D`:

- A scale of zero freezes the body: linear and angular velocity are zeroed, the Box2D body is disabled, the velocity fields are cleared and the body is marked frozen. A frozen entity neither integrates nor receives impulses.
- A positive scale thaws a frozen body: the Box2D body is re-enabled when the entity is also enabled, and the frozen mark is cleared.
- Authored impulses passed through `ApplyImpulse` are multiplied by the scale, and an impulse on a frozen entity is ignored.
- A missing component means a scale of one, so behaviour is unchanged until you author a clock.

What this combination gives you:

Effect	Global scale	Entity scale
Hit pause on one character	1	0 on that entity
Room-wide slow motion	0.25	1
Bullet time with the player exempt	0.25	4 on the player, to cancel the global slowdown
Full stop for a cutscene	0	1
Frozen scenery that never wakes	1	0

Two cautions. First, the time scale belongs to physics: animation, audio, UI and script timers keep their own clocks, so a stopped world can still animate a cursor or play music. Second, a frozen body is not the same as a sleeping body: sleeping is left to Box2D and preserves the body for a later wake, while a frozen body is disabled on purpose and stays disabled until its scale becomes positive.

9. The SDF primitive palette

The palette contains twenty exact signed distance families. A shape is authored in `SdfShapeComponent`, which wraps one `SdfShape`. The family selects which members the distance formula reads; every other member is ignored.

Family	Value	Members it reads	Palette default
Circle	0	`Radius`	radius 16
Box	1	`HalfExtents`	16 by 16
RoundedBox	2	`HalfExtents`, `Radius`	16 by 16, corner 4

		as corner radius	
Capsule	3	`HalfExtents` as half chord, `Radius` as thickness	chord 16, thickness 8
Segment	4	`HalfExtents` as half chord	chord 16
Ellipse	5	`HalfExtents` as the two radii	20 by 12
Rhombus	6	`HalfExtents` as half diagonals	18 by 12
EquilateralTriangle	7	`HalfExtents.Y` as circumradius	16
IsoscelesTriangle	8	`HalfExtents.X` half base, `HalfExtents.Y` half height	16 by 16
Trapezoid	9	`HalfExtents.X` bottom half width, `HalfExtents.Y` half height, `Width` top half width	16 by 12, top 8
Parallelogram	10	`HalfExtents`, `Width` as horizontal shear	16 by 12, shear 6
Hexagon	11	`HalfExtents.Y` as circumradius	16
Octagon	12	`HalfExtents.Y` as circumradius	16
Star	13	`HalfExtents.X` inner radius, `HalfExtents.Y` outer radius, `Sides` point count	inner 8, outer 18, 5 points
Pie	14	`Radius`, `Width` as half aperture	radius 16, half aperture 0.785398
Ring	15	`Radius` mid radius, `HalfExtents.X` half thickness	mid 16, thickness 4
Arc	16	`Radius` mid radius, `HalfExtents.X` half thickness, `Width` half aperture	mid 16, thickness 3, half aperture 0.785398
Cross	17	`HalfExtents` as the two arm half extents, `Radius` as corner radius	16 by 6
Moon	18	`Radius` outer radius, `HalfExtents.X` offset of the cutting circle, `HalfExtents.Y` cutting radius	outer 16, offset 8, cut 14
Heart	19	`Radius` as the overall size	16

Member rules worth knowing before authoring:

- `Angle` rotates the query around the origin in radians, so the shape rotates without moving its centre.
- `Width` is a secondary scalar whose meaning depends on the family: aperture for pie and arc, top half width for trapezoid, horizontal shear for parallelogram. Pie and arc clamp it to 0 through pi/2.

- `Sides` is read by Star only and clamps to 3 through 32.
- `Flags` is reserved for authoring and render hints and never changes the distance.
- The component is 32 bytes, trivially copyable and standard layout, and the palette is indexed by family, so `MakeSdfShape` returns a copy of the exact default row.

The three queries:

Function	Result
`SdfDistance`	Exact signed distance to the boundary, negative inside, positive outside
`SdfGradient`	Unit outward gradient from central differences with a 1e-3 step; zero where the difference vanishes
`SdfShapeCoversPoint`	True when the point is strictly inside

Every family is exact, including the ellipse, which evaluates the closed-form quartic root instead of iterating. An out-of-range family in a description returns the Circle row, so a corrupt type never produces a non-finite distance.

Using the palette with the rest of this manual:

- Build the shape from `SdfPrimitivePalette` or `MakeSdfShape`, then adjust the members for the family. Hand-rolling a distance function for a new shape is the wrong move: it breaks the palette contract, the inspector and the field bake.
- Complex collision rungs test the same field the SDF Visualizer draws. If a complex collider behaves strangely, look at the SDF Visualizer first.
- The palette is for collision shapes and diagnostic drawing. It is not a replacement for sprite art.

10. Optimisation playbook

Symptom	Where to look	The usual fix
Red sprites everywhere in Shader Complexity	Overlapping transparent quads	Cut, merge or mask layers; one background quad instead of several
Red only under a cluster of lamps	Light Complexity	Fewer, wider lights; mark decorative lamps Static
Static lamps still cost enough to see a frame difference	Cache counters	Fix a moving target or occluder that rebuilds the cache
Frame time spikes when a prop lands	Collision rungs	SimpleStatic for the floor, complex only where the polygon is wrong
Shadows disagree with the art	SDF Visualizer	Rebuild the field, tag the wall as an occluder
A trigger pushes the player	Collision Overlap	The rung is SimpleDynamic, not TriggerOnly
Every sprite is lit by every lamp	Radius and cone values	Shrink Radius, narrow ConeAngle of spot and arc lights
Many grey colliders in a level	NoCollision rung	Restore SimpleStatic where a wall exists

11. Settings, files and related documents

The two settings keys this manual depends on:

Key	Field	Default	Range	Meaning
`viewport_view_mode`	`viewportViewMode`	0	0 through 5, invalid values clamp to 0	The active mode, persisted in the project file and mirrored to the editor INI
`viewport_complexity_budget`	`viewportComplexityBudget`	8	1 through 64	The normalisation reference of the Shader Complexity heat ramp

Related documents:

- [Editor Settings Reference](EditorSettingsReference_EN.md): every editor key, its default, its clamp and its editor INI mirror.
- [Lighting](../../design/lighting.md): the CPU lighting path, SDF shadows and the GPU compute mask.
- [Time](../../design/time.md): clocks, timers and frame timing outside the physics step.
- [World systems](../../design/world-systems.md): tilemaps, skeletal IK and navigation, which consume the collision ladder and the render masks.
- [Project Settings Recommendations](ProjectSettingsRecommendations_EN.md): how the editor-side numbers fit a project profile.
- [Application Guide](FemboyEngineApplicationGuide_EN.md): the authoring workflow that uses these diagnostics.